

# Web Application Development

## Lecture 7: useEffect & Data Fetching

SWE 381 – Web Application Development

Dr. Ohoud Alharbi   Dr. Achraf Gazdar

Department of Software Engineering  
College of Computer and Information Sciences  
King Saud University

Term 2 – 2025/2026

# Learning Objectives

By the end of this lecture, you will be able to:

- Use `async/await` and `try/catch/finally` for asynchronous JavaScript
- Understand the **two-await fetch pattern** (response + JSON)
- Write `useEffect` with all three dependency array forms
- Apply **dependency array rules** to control when effects run
- Manage **loading / data / error states** for API calls
- Write **cleanup functions** to prevent memory leaks
- Fetch **multiple APIs in parallel** with `Promise.all`

# Agenda

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# Outline

- 1 **async / await + try/catch**
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# Promises and async/await – Quick Recap

## Why async/await?

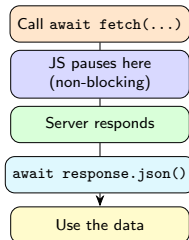
JavaScript is **single-threaded**. Fetching data from a server takes time – async/await lets us **wait for results** without blocking the browser, using cleaner syntax than raw Promise chains (`.then().catch()`).

## Promise chain (old style):

```
1 fetch('https://api.example.com/data')
2   .then(response => response.json())
3   .then(data => console.log(data))
4   .catch(error => console.error(error));
```

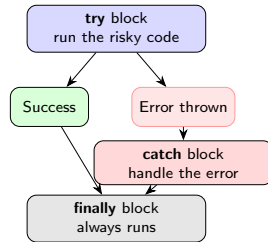
## async/await (modern, readable):

```
1 async function getData() {
2   try {
3     const response =
4       await fetch('https://api.example.com/data');
5     const data = await response.json();
6     console.log(data);
7   } catch (error) {
8     console.error('Failed:', error);
9   }
10 }
11 getData();
```



# try / catch / finally

```
1 async function fetchUser(id) {
2   try {
3     // 1. Attempt the risky operation
4     const response = await fetch(
5       'https://api.example.com/users/${id}'
6     );
7
8     // Not all errors throw automatically!
9     // 404 / 500 responses don't throw
10    if (!response.ok) {
11      throw new Error(
12        'HTTP error: ${response.status}'
13      );
14    }
15
16    const data = await response.json();
17    return data;
18  } catch (error) {
19    // 2. Handle ANY error (network or thrown)
20    console.error('Fetch failed:', error.message);
21    return null;
22  } finally {
23    // 3. Always runs -- success OR failure
24    console.log('Fetch attempt complete');
25  }
26 }
```



## 💡 finally for loading

finally is perfect for turning off a loading spinner – it runs whether the fetch succeeded or failed.

# Outline

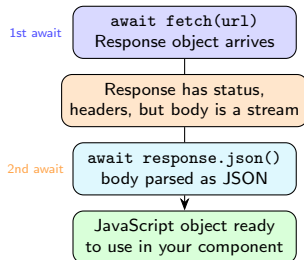
- 1 async / await + try/catch
- 2 Fetch API – Two Awaits**
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# The Fetch API and the Two-Await Pattern

## Why two awaits?

`fetch()` returns a Promise that resolves to a **Response object** (headers only). Reading the body is a **second async step** – `response.json()` also returns a Promise. You need to await both.

```
async function getPost() {  
  // AWAIT 1: wait for server to respond  
  // (headers arrive, body not yet read)  
  const response = await fetch(  
    'https://jsonplaceholder.typicode.com/posts/1'  
  );  
  
  // AWAIT 2: read and parse the body as JSON  
  const data = await response.json();  
  
  console.log(data);  
  // { id: 1, title: "...", body: "...", userId: 1 }  
}  
  
// WRONG -- missing second await:  
// const data = response.json(); // data is a Promise!  
// console.log(data.title);      // undefined -- no await  
// !
```



# fetch + response.ok Check

## fetch does NOT throw on HTTP errors

fetch() only rejects on **network failure** (e.g. no internet). HTTP error codes like 404 or 500 resolve successfully – you must check response.ok (or response.status) manually.

```
1 async function getUser(id) {
2   try {
3     const response = await fetch(
4       'https://jsonplaceholder.typicode.com/users/${id}'
5     );
6
7     // response.ok is true for status 200-299
8     if (!response.ok) {
9       throw new Error(`Error ${response.status}: ${response.statusText}`);
10    }
11
12    const data = await response.json();
13    return data;
14  } catch (error) {
15    console.error('Request failed:', error.message);
16    throw error; // re-throw so caller can handle
17  }
18 }
19
```

| Scenario         | response.ok | Promise rejects?                |
|------------------|-------------|---------------------------------|
| 200 OK           | true        | No                              |
| 404 Not Found    | false       | No – must check manually        |
| 500 Server Error | false       | No – must check manually        |
| No network       | n/a         | Yes – catch fires automatically |

# Outline

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 **useEffect Syntax & 3 Forms**
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# What is useEffect?

## W3Schools – React useEffect Hook

The `useEffect` Hook allows you to perform **side effects** in your components. Some examples of side effects are: fetching data, directly updating the DOM, and timers. `useEffect` accepts two arguments: a **callback function** and an optional **dependency array**.

### What are side effects?

- Fetching data from an API
- Setting the document title
- Setting up a timer (`setInterval`)
- Adding an event listener
- Subscribing to a WebSocket
- Reading from `localStorage`

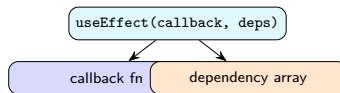


### Why not just call fetch directly?

Calling `fetch` at the top level of a component would re-run on **every render**. `useEffect` lets you control **exactly when** the side effect runs.

### General syntax:

```
1 import { useEffect } from 'react';
2
3 useEffect(
4   () => {
5     // side effect code here
6   },
7   [/* dependency array */]
8 );
```



# useEffect – Three Forms

## Form 1: No array

Runs after **every** render

```
useEffect(() => {  
  document.title =  
    'Count: ${count}';  
  // Runs after EVERY render  
  // Avoid -- can cause loops  
});
```

Rarely what you want.

## Form 2: Empty []

Runs **once** on mount

```
useEffect(() => {  
  // Like componentDidMount  
  // Runs ONCE when component  
  // first appears in the DOM  
  fetchData();  
}, []); // empty array
```

Most common for data fetch.

## Form 3: With deps

Runs when **deps** change

```
useEffect(() => {  
  // Runs on mount AND  
  // whenever userId or  
  // filter changes  
  fetchUser(userId);  
}, [userId, filter]);
```

Re-fetches on prop change.

| Form | Dependency array   | When effect runs                |
|------|--------------------|---------------------------------|
| 1    | No array (omitted) | After <b>every</b> render       |
| 2    | Empty []           | <b>Once</b> – on mount only     |
| 3    | [dep1, dep2, ...]  | On mount + when any dep changes |

# useEffect Form 2 – W3Schools Timer Example

## W3Schools – useEffect with empty array

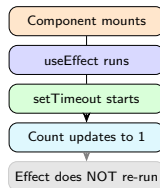
To fix the issue of running too often, let's only run this effect on the initial render by adding **empty brackets []** as the second argument.

```
1 import { useState, useEffect } from 'react';
2
3 function Timer() {
4   const [count, setCount] = useState(0);
5
6   useEffect(() => {
7     // Runs ONCE when Timer mounts
8     setTimeout(() => {
9       setCount(c => c + 1);
10     }, 1000);
11   }, []); // <-- empty array: mount only
12
13   return (
14     <h1>I rendered {count} times!</h1>
15   );
16 }
```

```
1 // Form 3: re-runs when 'count' changes
2 useEffect(() => {
3   setTimeout(() => {
4     setCount(c => c + 1);
5   }, 1000);
6 }, [count]); // re-runs every second
```

⌚ After 1 second

I rendered **1** times!



# Outline

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules**
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# Dependency Array – The Rules

## W3Schools – useEffect dependencies

We should always include the second parameter (dependency array). If there are multiple dependencies, they should be included in the array.

### Rule: include everything you USE inside the effect:

```
1 function UserProfile({ userId, filter }) {  
2   const [user, setUser] = useState(null);  
3  
4   useEffect(() => {  
5     // Uses: userId and filter  
6     // Both must be in the array!  
7     fetchUser(userId, filter)  
8       .then(data => setUser(data));  
9   }, [userId, filter]); // correct  
10  
11   // WRONG -- missing 'filter':  
12   // }, [userId]);  
13   // If filter changes, effect won't re-run  
14   // -- you'll see stale data!  
15 }
```

```
1 // Multiple deps: runs if ANY changes  
2 useEffect(() => {  
3   loadData(page, query, sort);  
4 }, [page, query, sort]);
```

### What to include:

- ✓ Props used inside the effect
- ✓ State used inside the effect
- ✓ Variables from component scope
- ✗ State **setter** functions (stable)
- ✗ Variables defined **inside** the effect
- ✗ Constants defined **outside** the component

### ⚠ Infinite loop danger

If you update state inside `useEffect` and include that state in deps – the effect re-runs, updates state again, loops forever. Use `[]` or re-design.

# Dependency Array – W3Schools Counter Example

```
1 import { useState, useEffect } from 'react';
2
3 function Counter() {
4   const [count, setCount] = useState(0);
5   const [color, setColor] = useState("blue");
6
7   // Runs on mount AND when 'count' changes
8   useEffect(() => {
9     document.title = `Count: ${count}`;
10  }, [count]);
11
12  // Runs on mount AND when 'color' changes
13  useEffect(() => {
14    document.body.style.background = color;
15  }, [color]);
16
17  return (
18    <>
19      <h1 style={{ color }}>
20        Count: {count}
21      </h1>
22      <button onClick={() =>
23        setCount(c => c + 1)}>
24        Increment
25      </button>
26      <button onClick={() =>
27        setColor("red")}>
28        Red
29      </button>
30    </>
31  );
32 }
```

After clicking Increment 3x

Count: 3

Increment

Red

*document.title = "Count: 3"*  
*background unchanged*

Two separate effects = two separate jobs:

Effect 1 runs when count changes

Effect 2 runs when color changes

React runs them independently

💡 One effect per concern

Split unrelated side effects into **separate** `useEffect` calls for clarity and correct dependency tracking.

# Outline

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States**
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# The Three Data-Fetching States

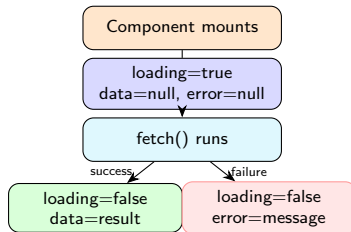
Every real API call needs to handle three possible states:

- 1 **Loading** – request is in flight, show a spinner
- 2 **Data** – request succeeded, render the data
- 3 **Error** – request failed, show an error message

## State setup

Use three separate `useState` variables:

- `data` – initially `null`
- `loading` – initially `true`
- `error` – initially `null`



# Loading / Data / Error – Implementation

```
1 import { useState, useEffect } from 'react';
2
3 function PostList() {
4   const [data, setData] = useState(null);
5   const [loading, setLoading] = useState(true); // start loading immediately
6   const [error, setError] = useState(null);
7
8   useEffect(() => {
9     const fetchData = async () => {
10       try {
11         const response = await fetch(
12           'https://jsonplaceholder.typicode.com/posts'
13         );
14         if (!response.ok) throw new Error(`HTTP ${response.status}`);
15         const json = await response.json();
16         setData(json);
17       } catch (err) {
18         setError(err.message);
19       } finally {
20         setLoading(false); // always turn off loading
21       }
22     };
23     fetchData();
24   }, []); // empty array: fetch once on mount
25
26   // 3 render cases -- loading, error, or data
27   if (loading) return <p>Loading posts...</p>;
28   if (error) return <p>Error: {error}</p>;
29   return (
30     <ul>
31       {data.map(post => (
32         <li key={post.id}>{post.title}</li>
33       ))}
34     </ul>
35   );
36 }
```

# Loading / Data / Error – Browser States

## Loading state

*Loading posts...*

```
// Render 1 (immediate):  
loading = true  
data = null  
error = null
```

```
// Shows:  
<p>Loading posts...</p>
```

## Data state

- Post title 1
- Post title 2
- Post title 3
- ...

```
1 // After success:  
2 loading = false  
3 data = [...posts]  
4 error = null  
5  
6 // Shows:  
7 <ul>...</ul>
```

## Error state

Error: HTTP 404

```
1 // After failure:  
2 loading = false  
3 data = null  
4 error = "HTTP 404"  
5  
6 // Shows:  
7 <p>Error: HTTP 404</p>
```



## Use finally for clean transitions

Always put `setLoading(false)` inside `finally` – not in `try` and `catch` separately. This guarantees loading always turns off regardless of outcome.

# Re-fetching on Prop Change

```
1 import { useState, useEffect } from 'react';
2
3 function UserDetails({ userId }) {
4   const [user, setUser] = useState(null);
5   const [loading, setLoading] = useState(true);
6   const [error, setError] = useState(null);
7
8   useEffect(() => {
9     // Runs on mount AND when userId changes
10    setLoading(true);
11    setError(null);
12
13    const fetchUser = async () => {
14      try {
15        const res = await fetch(
16          'https://jsonplaceholder.typicode.com'
17          + '/users/${userId}'
18        );
19        if (!res.ok) throw new Error('Not found');
20        const json = await res.json();
21        setUser(json);
22      } catch (err) {
23        setError(err.message);
24      } finally {
25        setLoading(false);
26      }
27    };
28    fetchUser();
29
30    }, [userId]); // re-fetch when userId changes
31
32    if (loading) return <p>Loading user {userId}...</p>;
33    if (error) return <p>Error: {error}</p>;
34    return <h2>{user?.name} ({user?.email})</h2>;
35  }
```

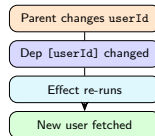
🔍 userId=1

**Leanne Graham**  
(Sincere@april.biz)

🔍 userId=2 (after prop change)

*Loading user 2...*

**Ervin Howell**  
(Shanna@melissa.tv)



# Outline

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions**
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# Why Cleanup is Necessary

## W3Schools – useEffect cleanup

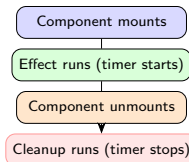
Some effects require cleanup to reduce memory leaks. Timeouts, subscriptions, event listeners, and other effects that are no longer needed should be disposed. We do this by including a **return function** at the end of the useEffect Hook.

### The problem – without cleanup:

```
1 function Timer() {  
2   const [count, setCount] = useState(0);  
3  
4   useEffect(() => {  
5     // Interval keeps running even after  
6     // the component unmounts!  
7     const id = setInterval(() => {  
8       setCount(c => c + 1);  
9     }, 1000);  
10    // No cleanup = memory leak + error:  
11    // "Can't perform state update on  
12    // unmounted component"  
13  }, []);  
14  
15  return <p>Count: {count}</p>;  
16 }
```

### The fix – with cleanup:

```
1 useEffect(() => {  
2   const id = setInterval(() => {  
3     setCount(c => c + 1);  
4   }, 1000);  
5  
6   // Return a cleanup function!  
7   return () => {  
8     clearInterval(id); // stop timer  
9   };  
10 }, []);
```



# Cleanup Function – Syntax

## Syntax skeleton:

```
1 useEffect(() => {  
2   // ---- SETUP code ----  
3   // (runs after render / when deps change)  
4  
5   return () => {  
6     // ---- CLEANUP code ----  
7     // (runs BEFORE next effect or unmount)  
8   };  
9 }, [dependencies]);
```

## Event listener cleanup:

```
1 useEffect(() => {  
2   const handleResize = () => {  
3     setWidth(window.innerWidth);  
4   };  
5   window.addEventListener('resize', handleResize);  
6  
7   return () => {  
8     window.removeEventListener(  
9       'resize', handleResize  
10    );  
11  };  
12 }, []);
```

## Fetch abort cleanup:

```
1 useEffect(() => {  
2   const controller = new AbortController();  
3  
4   const fetchData = async () => {  
5     try {  
6       const res = await fetch(url, {  
7         signal: controller.signal  
8       });  
9       const json = await res.json();  
10      setData(json);  
11    } catch (err) {  
12      if (err.name !== 'AbortError')  
13        setError(err.message);  
14    }  
15  };  
16  fetchData();  
17  
18  return () => {  
19    controller.abort(); // cancel fetch  
20  };  
21 }, [url]);
```

# When Do You Need a Cleanup Function?

## Always clean up:

- ⚠ `setInterval` / `setTimeout`  
Use `clearInterval` / `clearTimeout`
- ⚠ `addEventListener`  
Use `removeEventListener`
- ⚠ `fetch` with `AbortController`  
Use `controller.abort()`
- ⚠ `WebSocket` subscriptions  
Use `socket.close()`
- ⚠ External library subscriptions  
Call the unsubscribe function

## No cleanup needed for:

- ✓ Simple state updates
- ✓ `document.title` updates
- ✓ Console logging
- ✓ One-time data fetch (no abort needed)

### Cleanup timeline

#### Cleanup runs:

- Just **before** the effect re-runs (deps changed)
- When the component is **unmounted**

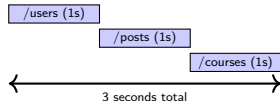
# Outline

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching**
- 8 Complete Example & Summary
- 9 Recap

# Sequential vs Parallel Fetching

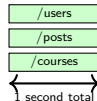
## Sequential (slow):

```
// Each waits for the previous  
// Total time = 1s + 1s + 1s = 3 seconds  
const users    = await fetch('/users');  
const posts    = await fetch('/posts');  
const courses  = await fetch('/courses');
```



## Parallel with Promise.all (fast):

```
1 // All three start AT THE SAME TIME  
2 // Total time = max(1s, 1s, 1s) = 1 second  
3 const [users, posts, courses] =  
4   await Promise.all([  
5     fetch('/users'),  
6     fetch('/posts'),  
7     fetch('/courses')  
8   ]);
```



# Promise.all in useEffect

```
1 import { useState, useEffect } from 'react';
2
3 function Dashboard() {
4   const [users, setUsers] = useState([]);
5   const [posts, setPosts] = useState([]);
6   const [loading, setLoading] = useState(true);
7   const [error, setError] = useState(null);
8
9   useEffect(() => {
10     const fetchAll = async () => {
11       try {
12         // All requests fire at the same time
13         const [usersRes, postsRes] = await Promise.all([
14           fetch('https://jsonplaceholder.typicode.com/users'),
15           fetch('https://jsonplaceholder.typicode.com/posts')
16         ]);
17
18         // Parse both JSON responses (also in parallel!)
19         const [usersData, postsData] = await Promise.all([
20           usersRes.json(),
21           postsRes.json()
22         ]);
23
24         setUsers(usersData);
25         setPosts(postsData);
26
27       } catch (err) {
28         // If ANY request fails, catch fires
29         setError(err.message);
30       } finally {
31         setLoading(false);
32       }
33     };
34     fetchAll();
35   }, []);
36
37   if (loading) return <p>Loading dashboard...</p>;
38   if (error) return <p>Error: {error}</p>;
39   return (
40     <div>
41       <p>Users: {users.length}</p>
42       <p>Posts: {posts.length}</p>
43     </div>
44   );
45 }
```

# Promise.all – Output and Error Behaviour

## Successful parallel fetch

Loading dashboard...

*(after 1 second)*

Users: **10**

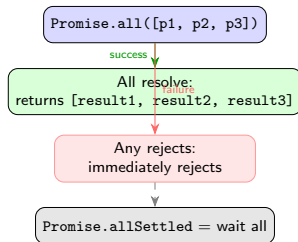
Posts: **100**

## If either request fails

Error: HTTP 503

## ! Promise.all fail-fast

If **any** promise in the array rejects, `Promise.all` immediately rejects. The other results are discarded. Use `Promise.allSettled` if you want all results regardless of failures.



# Outline

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary**
- 9 Recap

# Complete Data-Fetching Component

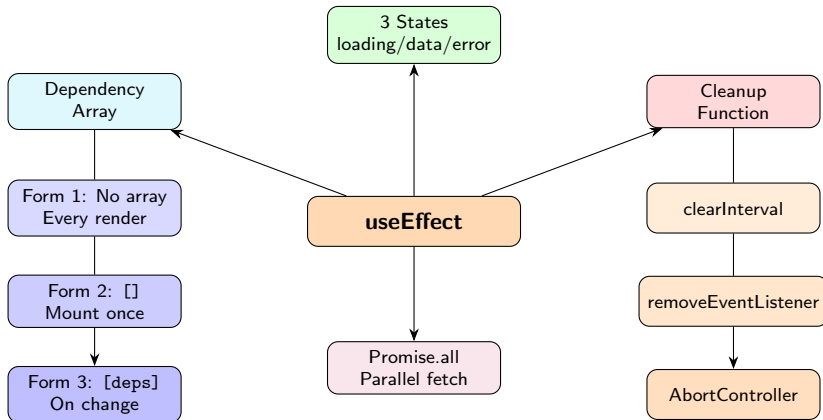
## Data layer (state + effect):

```
1 import { useState, useEffect } from 'react';
2
3 const API = 'https://jsonplaceholder.typicode.com';
4
5 function CourseExplorer({ deptId }) {
6   const [courses, setCourses] = useState([]);
7   const [loading, setLoading] = useState(true);
8   const [error, setError] = useState(null);
9   const [search, setSearch] = useState('');
10
11   useEffect(() => {
12     const controller = new AbortController();
13     const fetchCourses = async () => {
14       setLoading(true);
15       setError(null);
16       try {
17         const res = await fetch(
18           `${API}/posts?userId=${deptId}`,
19           { signal: controller.signal }
20         );
21         if (!res.ok)
22           throw new Error(`Error ${res.status}`);
23         const data = await res.json();
24         setCourses(data);
25       } catch (err) {
26         if (err.name !== 'AbortError')
27           setError(err.message);
28       } finally {
29         setLoading(false);
30       }
31     };
32   });
33 }
```

## Render layer (derived + JSX):

```
1 // Derived -- no extra state needed
2 const filtered = courses.filter(c =>
3   c.title.toLowerCase().includes(
4     search.toLowerCase()
5   )
6 );
7
8 if (loading) return <p>Loading...</p>;
9 if (error) return <p>Error: {error}</p>;
10 return (
11   <div>
12     <input
13       value={search}
14       onChange={e => setSearch(e.target.value)}
15       placeholder="Search courses..."
16     />
17     <p>
18       {filtered.length} of {courses.length}
19     </p>
20     <ul>
21       {filtered.map(c => (
22         <li key={c.id}>{c.title}</li>
23       ))}
24     </ul>
25   </div>
26 );
27 }
```

# useEffect Data-Fetching – Concept Map



# Common Mistakes – Quick Reference

## Mistake 1 – async useEffect:

```
1 // WRONG -- useEffect cannot be async!
2 useEffect(async () => {
3   const data = await fetch(...);
4 }, []);
5
6 // CORRECT -- async function INSIDE
7 useEffect(() => {
8   const load = async () => {
9     const data = await fetch(...);
10   };
11   load();
12 }, []);
```

## Mistake 2 – missing response.ok check:

```
1 // WRONG -- 404 won't throw!
2 const res = await fetch(url);
3 const data = await res.json();
4
5 // CORRECT
6 const res = await fetch(url);
7 if (!res.ok) throw new Error(res.status);
8 const data = await res.json();
```

## Mistake 3 – missing deps:

```
1 // WRONG -- userId used but not in deps
2 useEffect(() => {
3   fetchUser(userId);
4 }, []); // stale data!
5
6 // CORRECT
7 useEffect(() => {
8   fetchUser(userId);
9 }, [userId]);
```

## Mistake 4 – no cleanup:

```
1 // WRONG -- timer leaks!
2 useEffect(() => {
3   setInterval(fn, 1000);
4 }, []);
5
6 // CORRECT
7 useEffect(() => {
8   const id = setInterval(fn, 1000);
9   return () => clearInterval(id);
10 }, []);
```

# Outline

- 1 async / await + try/catch
- 2 Fetch API – Two Awaits
- 3 useEffect Syntax & 3 Forms
- 4 Dependency Array Rules
- 5 loading / data / error States
- 6 Cleanup Functions
- 7 Promise.all – Parallel Fetching
- 8 Complete Example & Summary
- 9 Recap

# Lecture 7 – Key Takeaways

- ❶ **async/await** makes asynchronous code readable; always wrap in `try/catch/finally`
- ❷ **fetch** requires two awaits: one for the Response, one for `.json()`; check `response.ok` manually
- ❸ `useEffect` runs side effects after render; never make the callback itself `async`
- ❹ **Three forms**: no array (every render), `[]` (mount only), `[deps]` (mount + on change)
- ❺ **Dependency array**: include every prop and state used inside the effect – omissions cause stale data
- ❻ Manage **three states**: loading, data, error – use `finally` to turn off loading
- ❼ **Cleanup functions** (returned from `useEffect`) prevent memory leaks from timers, listeners, open fetches
- ❽ `Promise.all` runs multiple fetches in parallel – faster than sequential awaits
- ❾ If *any* promise in `Promise.all` rejects, the whole thing rejects – use `AbortController` for cleanup

**Next Lecture:** React Context API – Global State without Prop Drilling

# Practice Exercises

- 1 Write an `async` function using `try/catch/finally` that fetches `https://jsonplaceholder.typicode.com/posts/1` and logs the title. Confirm the two-await pattern: first for the response, second for `.json()`.
- 2 Create a component that uses `useEffect` with an **empty dependency array** to fetch a list of users from JSONPlaceholder on mount. Display “Loading...” while fetching and the list once loaded.
- 3 Add **error handling**: force a 404 by fetching a bad URL (`/users/9999`). Show “Error: ...” in the UI using an error state. Use `response.ok` to detect the failure.
- 4 Build a **UserDetail** component that accepts a `userId` prop and re-fetches when it changes. Add three buttons in the parent to switch between users 1, 2, and 3.
- 5 Add a **cleanup function** using `AbortController` to the component from Exercise 4. Confirm that rapidly switching users does not cause stale state updates.
- 6 Implement a **countdown timer** using `setInterval` inside `useEffect`. Add a cleanup that calls `clearInterval` when the component unmounts. Toggle the timer component on and off to test the cleanup.
- 7 **Challenge**: Build a **Dashboard** component that uses `Promise.all` to fetch users and posts **in parallel**. Show a single loading state, render both datasets side by side once loaded, and handle any API error.